Homework (Simulation)

This program, x86.py, allows you to see how different thread interleavings either cause or avoid race conditions. See the README for details on how the program works and answer the questions below.

Questions

1. Examine flag.s. This code “implements” locking with a single memory flag. Can you understand the assembly?

flag == 0: lock is empty.
flag == 1: lock is being used.

Flow
1. Try acquiring a lock by reading a flag, testing, and moving the action.
2. Critical section reads count, increments 1, and saves it again.
3. Release lock
4. Repeat

Seems like not a stable lock.
Reading a flag and using a 1 are not atomic. Two threads can see flag==0 and run mov $1, flag. And this can lead to race condition.

flag.s

```
.var flag
.var count

.main
.top

.acquire
mov  flag, %ax      # get flag
test $0, %ax        # if we get 0 back: lock is free!
jne  .acquire       # if not, try again
mov  $1, flag       # store 1 into flag

# critical section
mov  count, %ax     # get the value at the address
add  $1, %ax        # increment it
mov  %ax, count     # store it back

# release lock
mov  $0, flag       # clear the flag now

# see if we're still looping
sub  $1, %bx
test $0, %bx
jgt .top

halt
```

2. When you run with the defaults, does flag.s work? Use the -M and -R flags to trace variables and registers (and turn on -c to see their values). Can you predict what value will end up in flag?

python3 x86.py -p flag.s -t 2 -a bx=1,bx=1 -M flag,count -R ax,bx -c

```
[cssuwbstudent@css430 threads-locks]$ python3 x86.py -p flag.s -t 2 -a bx=1,bx=1
 -M flag,count -R ax,bx

 flag count      ax    bx          Thread 0                Thread 1

    ?     ?       ?     ?
    ?     ?       ?     ?   1000 mov  flag, %ax
    ?     ?       ?     ?   1001 test $0, %ax
    ?     ?       ?     ?   1002 jne  .acquire
    ?     ?       ?     ?   1003 mov  $1, flag
    ?     ?       ?     ?   1004 mov  count, %ax
    ?     ?       ?     ?   1005 add  $1, %ax
    ?     ?       ?     ?   1006 mov  %ax, count
    ?     ?       ?     ?   1007 mov  $0, flag
    ?     ?       ?     ?   1008 sub  $1, %bx
    ?     ?       ?     ?   1009 test $0, %bx
    ?     ?       ?     ?   1010 jgt .top
    ?     ?       ?     ?   1011 halt
    ?     ?       ?     ?   ----- Halt;Switch -----  ----- Halt;Switch -----
    ?     ?       ?     ?                            1000 mov  flag, %ax
    ?     ?       ?     ?                            1001 test $0, %ax
    ?     ?       ?     ?                            1002 jne  .acquire
    ?     ?       ?     ?                            1003 mov  $1, flag
    ?     ?       ?     ?                            1004 mov  count, %ax
    ?     ?       ?     ?                            1005 add  $1, %ax
    ?     ?       ?     ?                            1006 mov  %ax, count
    ?     ?       ?     ?                            1007 mov  $0, flag
    ?     ?       ?     ?                            1008 sub  $1, %bx
    ?     ?       ?     ?                            1009 test $0, %bx
    ?     ?       ?     ?                            1010 jgt .top
    ?     ?       ?     ?                            1011 halt
```

```
[cssuwbstudent@css430 threads-locks]$ python3 x86.py -p flag.s -t 2 -a bx=1,bx=1
 -M flag,count -R ax,bx -c

 flag count      ax    bx          Thread 0                Thread 1

    0     0       0     1
    0     0       0     1   1000 mov  flag, %ax
    0     0       0     1   1001 test $0, %ax
    0     0       0     1   1002 jne  .acquire
    1     0       0     1   1003 mov  $1, flag
    1     0       0     1   1004 mov  count, %ax
    1     0       1     1   1005 add  $1, %ax
    1     1       1     1   1006 mov  %ax, count
    0     1       1     1   1007 mov  $0, flag
    0     1       1     0   1008 sub  $1, %bx
    0     1       1     0   1009 test $0, %bx
    0     1       1     0   1010 jgt .top
    0     1       1     0   1011 halt
    0     1       0     1   ----- Halt;Switch -----  ----- Halt;Switch -----
    0     1       0     1                            1000 mov  flag, %ax
    0     1       0     1                            1001 test $0, %ax
    0     1       0     1                            1002 jne  .acquire
    1     1       0     1                            1003 mov  $1, flag
    1     1       1     1                            1004 mov  count, %ax
    1     1       2     1                            1005 add  $1, %ax
    1     2       2     1                            1006 mov  %ax, count
    0     2       2     1                            1007 mov  $0, flag
    0     2       2     0                            1008 sub  $1, %bx
    0     2       2     0                            1009 test $0, %bx
    0     2       2     0                            1010 jgt .top
    0     2       2     0                            1011 halt
```

It appears to work.
Initial value: flag=0, count=0
Thread 0 completes its critical section first, then thread 1 runs. The first values are count = 2 and flag = 0. So the value that ends up in the flag is 0 because the lock is released.

3. Change the value of the register %bx with the -a flag (e.g., -a bx=2,bx=2 if you are running just two threads). What does the code do? How does it change your answer for the question above?

python3 x86.py -p flag.s -t 2 -a bx=2,bx=2 -M flag,count -R ax,bx -c
Output analysis:
Each thread runs the loop twice, so the critical section is executed four times in total (2 threads x 2 loops = 4).
This time, the final values are count = 4 and flag = 0.
Code needs more lock acquire and release attempts, but it is still working with the default scheduling.
Compared to Q2, the final value is still 0.

4. Set bx to a high value for each thread, and then use the -i flag to generate different interrupt frequencies; what values lead to a bad outcomes? Which lead to good outcomes?

```
python3 x86.py -p flag.s -t 2 -a bx=100,bx=100 -M flag,count -c
python3 x86.py -p flag.s -t 2 -a bx=100,bx=100 -M flag,count -c -i 1
python3 x86.py -p flag.s -t 2 -a bx=100,bx=100 -M flag,count -c -i 2
python3 x86.py -p flag.s -t 2 -a bx=100,bx=100 -M flag,count -c -i 3
```

A good outcome is when the final count matches the expected total increments.
A bad outcome is when the final count is lower than expected. It showed lost updates due to race conditions.
These bad cases happened when interrupts occur between checking the flag and setting it because it allowed both threads into the critical section.

5. Now let's look at the program test-and-set.s. First, try to understand the code, which uses the xchg instruction to build a simple locking primitive. How is the lock acquire written? How about lock release?

Lock acquire is implemented with an atomic xchg. The thread sets %ax to 1, executes xch %ax, mutex, and then checks %ax. If %ax is 0, it acquired the lock. Otherwise, it retires in .acquire. Lock release is simply mov $0, mutex.

test-and-set.s

```
.var mutex
.var count

.main
.top

.acquire
mov  $1, %ax
xchg %ax, mutex     # atomic swap of 1 and mutex
test $0, %ax        # if we get 0 back: lock is free!
jne  .acquire       # if not, try again

# critical section
mov  count, %ax     # get the value at the address
add  $1, %ax        # increment it
mov  %ax, count     # store it back

# release lock
mov  $0, mutex

# see if we're still looping
sub  $1, %bx
test $0, %bx
jgt .top

halt
```

6. Now run the code, changing the value of the interrupt interval (-i) again, and making sure to loop for a number of times. Does the code always work as expected? Does it sometimes lead to an inefficient use of the CPU? How could you quantify that? 정확성 + 비효율성

python3 x86.py -p test-and-set.s -t 2 -a bx=100,bx=100 -M mutex,count -c -i [#] 1,2,5,10,50

The code always produced count = 200, so it worked as expected.
It has a CPU inefficiency due to busy-waiting when a thread cannot acquire the lock. It repeated executing the acquire loop without making progress.

7. Use the -P flag to generate specific tests of the locking code. For example, run a schedule that grabs the lock in the first thread, but then tries to acquire it in the second. Does the right thing happen? What else should you test?

python3 x86.py -p test-and-set.s -t 2 -a bx=1,bx=1 -M mutex,count -R ax,bx -c -P 000111111000

Above is an intended call where thread 0 acquires the lock, but thread 1 attempts to acquire it. The trace shows thread 1 repeatedly failing in the acquire loop while mutex=1, so it doesn't enter the critical section early. After thread 0 releases the lock, thread 1 acquires it and proceeds. Final values are count = 2 and mutex = 0. Other possible tests are long hold by one thread while the other spins or larger loop counts.

8. Now let's look at the code in peterson.s, which implements Peterson's algorithm (mentioned in a sidebar in the text). Study the code and see if you can make sense of it.

Each thread uses flag[self] to announce intent to enter the critical section, and sets turn = 1-self to

give the other thread priority in case of conflict. The spin loop waits and exits when either the other thread is not interested or it is this thread's turn. Then it enters the critical section and is released by setting flag[self] = 0.

9. Now run the code with different values of -i. What kinds of different behavior do you see? Make sure to set the thread IDs appropriately (using -a bx=0,bx=1 for example) as the code assumes it.

python3 x86.py -p peterson.s -t 2 -a bx=0,bx=1 -M count,turn,flag -R ax,bx,cx -c -i [#] #1,5,50

With i 1, there is heavy interleaving and more spinning in the wait loop.
With i 5, interleaving is moderate.
With i 50, execution is mostly sequential with little spinning.
All runs had a final count of 2.
Changing -i affects how much waiting/spinning occurs, not mutex correctness.

10. Can you control the scheduling (with the -P flag) to "prove" that the code works? What are the different cases you should hold? Think about mutual exclusion and deadlock avoidance.

Yes, using -P allows controlled scheduling. Possible different cases to handle are mutex, deadlock avoidance, or single-thread. In test cases, the final count is still 2, and both threads halt successfully. They support mutex and deadlock avoidance. It became evident that the controlled schedules with -P work correctly.

11. Now study the code for the ticket lock in ticket.s. Does it match the code in the chapter? Then run with the following flags: -a bx=1000,bx=1000 (causing each thread to loop through the critical section 1000 times). Watch what happens; do the threads spend much time spin-waiting for the lock?

python3 x86.py -p ticket.s -t 2 -a bx=1000,bx=1000 -M ticket,turn,count -R ax,bx,cx -c

STATS:: Instructions 99463
STATS:: Emulation Rate 81.17 kinst/sec

ticket.s matches the chapter's ticket-lock algorithm. The acquire uses fetchadd on the ticket and spins until turn == myticket, and release increments turn with fetchadd. However, the thread spends substantial time spin-waiting: the trace repeatedly shows the .tryagain loop. The stat also shows a high instruction count. It does show the CPU overhead of busy waiting under contention.

12. How does the code behave as you add more threads?

Ticket lock remains correct and fair with more threads, but lock contention causes increasing busy-wait overhead like higher instruction count and lower efficiency.

13. Now examine yield.s, in which a yield instruction enables one thread to yield control of the CPU (realistically, this would be an OS primitive, but for simplicity, we assume an instruction does the task).

Find a scenario where test-and-set.s wastes cycles spinning, but yield.s does not. How many instructions are saved? In what scenarios do these savings arise?

python3 x86.py -p test-and-set.s -t 2 -a bx=1000,bx=1000 -c -S
python3 x86.py -p yield.s -t 2 -a bx=1000,bx=1000 -c -S

The Test-and-set wastes many cycles in the acquire loop. Yield.s avoids this by executing yield after a failed lock attempt. And this lets the other runnable thread make progress instead of burning CPU in busy-wait.

My run shows two STATS.
test-and-set.s: 33954 instructions
yield.s: 23334 instructions
Saved instructions: 10620

These savings arise when lock contention is high, critical sections are short, and a single thread repeatedly attempts to acquire the lock. In these cases, using yield can significantly reduce wasted instructions.

14. Finally, examine test-and-test-and-set.s. What does this lock do? What kind of savings does it introduce as compared to test-and-set.s?

test-and-test-and-set.s is a spin lock that first polls with normal loads and only uses atomic xchg when the lock appears free. It reduces wasted atomic operations vs. pain test-and-set, especially under contention.